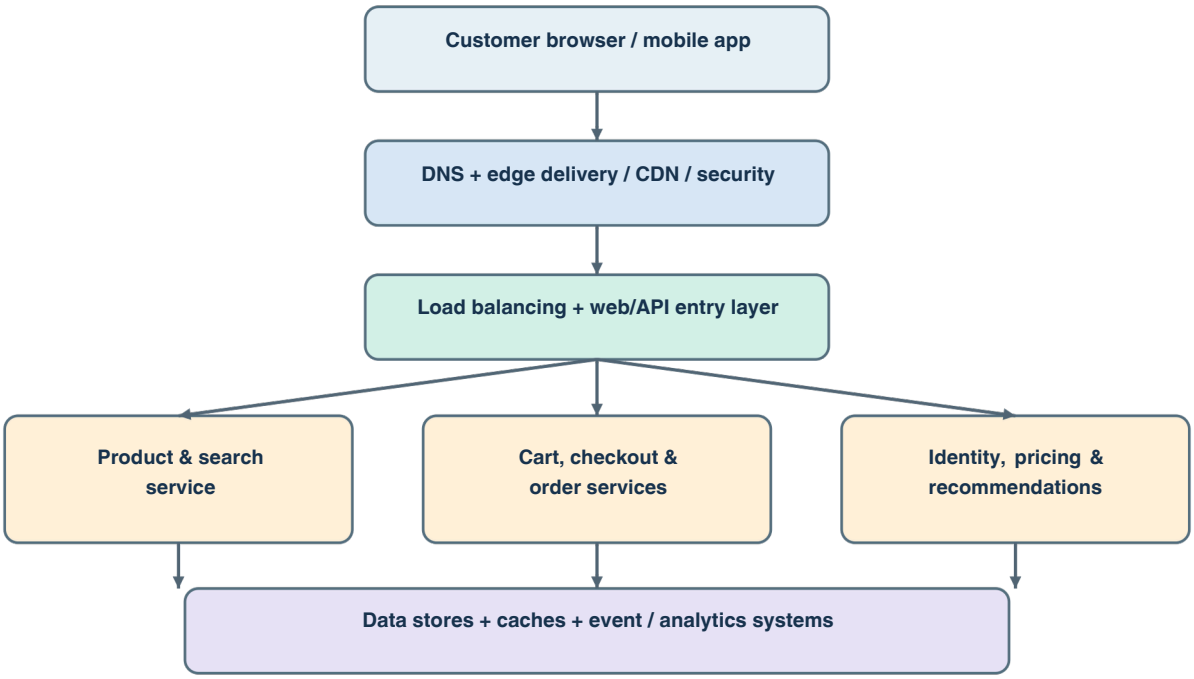
WEEK 4 ASSIGNMENT

Web Application Architecture Case Study: Amazon.com

A study of the architecture patterns that support a large-scale e-commerce platform

Illustrative logical architecture (not Amazon's proprietary production topology)



Objective

This report examines how a global e-commerce application can deliver product discovery, customer accounts, shopping carts, checkout and order management at scale. It uses public Amazon and AWS engineering material to explain the architectural patterns associated with Amazon.com. Some implementation details are proprietary and are not publicly documented as one complete technology stack; therefore, the diagram is a conceptual model, not an exact internal deployment map.

Prepared by: Shaam.R **Class / ID: DVP1112**

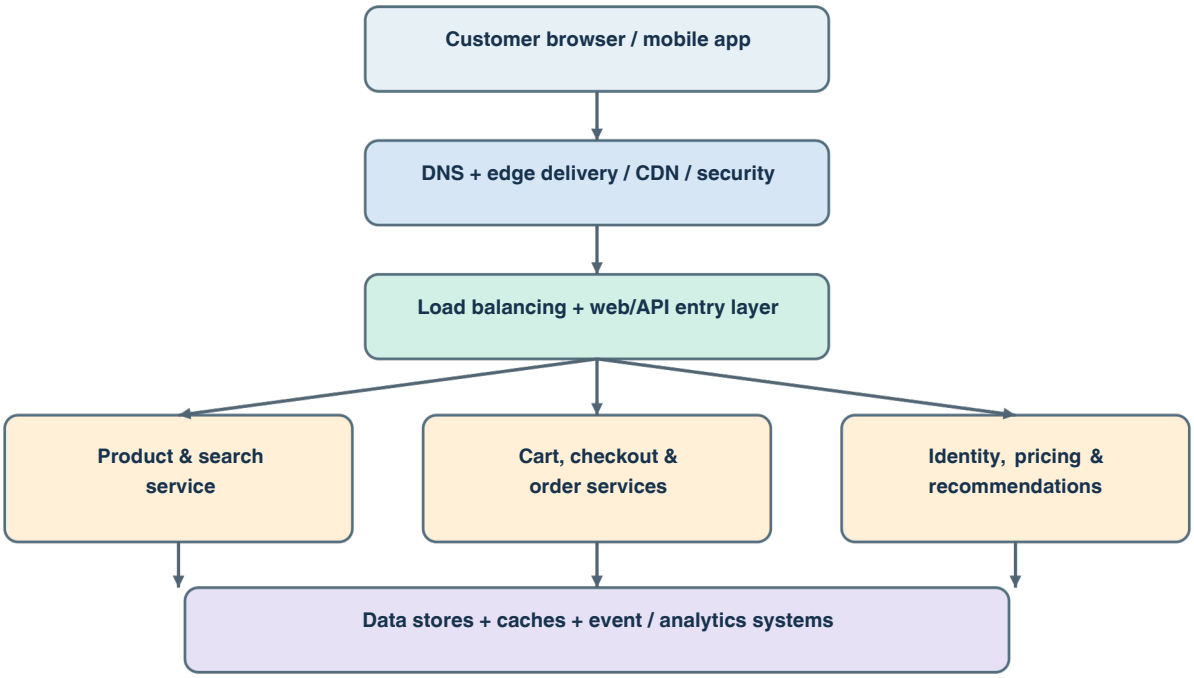
1. Application Overview and Architecture Type

Amazon.comis anonline retail platformthat combines product catalogues,search,recommendations, customer identity, pricing, inventory, shopping carts, payments, orders, delivery information and customer support. These capabilities must work together while handling different traffic patterns: product pages may receive many reads, while checkout and order updates require careful consistency and security.

Architecture type: distributed, service-oriented / microservices-style

The most useful high-level model is a distributed architecture with many independently owned services rather than a simple two-tier website. Public Amazon engineering material describes thousands of services and core systems that support Amazon Stores. In a microservices design, each service focuses on a business capability and can be deployed or scaled separately. Amazon's exact service boundaries and the complete Amazon.com production topology are not public, so this report uses a representative architecture.

Illustrative logical architecture (not Amazon's proprietary production topology)



Request flow: a customer opens Amazon.com; DNS and edge delivery route the request; web/API entry points send it to the relevant business service; the service reads or updates data and may communicate with other services; the response returns to the customer. Asynchronous events can support downstream work such as notifications, analytics or fulfilment workflows.

Why this structure helps

• Independent scaling: product browsing can scale differently from checkout.
• Fault isolation: a problem in one capability need not automatically stop every other capability.
• Team autonomy: teams can release and operate services on separate schedules.
• Technology flexibility: teams can select tools appropriate to their workload, while maintaining agreed interfaces and operational standards.

2. Web Server, Backend Technology and Tool Choices

Webserver / requestentrylayer

Amazon does not publish one definitive web-server product that can accurately be described as the sole server for all of Amazon.com. A large site can use edge delivery, load balancers, web servers, API front doors and internal service-to-service networking together. Therefore, it would be misleading to state that the entire site uses only Nginx or Apache. For this case study, the web-server role is described as a distributed web/API entry layer, with the exact production server implementation not publicly confirmed in the cited sources.

Backend technology

Amazon's public engineering material shows a broad, evolving technology environment rather than a single language or framework for the entire retail site. The eCommerce Foundation organisation says it operates core systems, infrastructure and services used by teams across Amazon Stores. Public documentation also describes AWS building blocks commonly used for e-commerce systems, including APIs, compute services, databases, caching and event-driven integrations. The precise language/framework mix for each Amazon.com service is not fully disclosed in one official public inventory.

For a student report, distinguish confirmed architecture patterns from examples of technologies that can implement them:

Layer	Role	Technology notes
Web / edge	Route requests, serve static assets, protect entry points	CDN, DNS, load balancing and web/API front door; exact Amazon.com web-server product not publicly confirmed
Application services	Search, catalogue, cart, checkout, identity, orders	Service-oriented components; implementation language varies by service
Data and events	Persist orders/catalogue data; notify downstream systems	Databases, caches, queues/event systems selected by workload
Operations	Deploy, monitor, scale and recover	Automation, metrics, alarms, capacity planning and resilience practices

Why these choices?

- Performance: caching and edge delivery reduce repeated work and long-distance transfers.
- Scalability: independent services can scale according to demand.
- Reliability: multiple instances, isolation and carefully managed dependencies reduce single points of failure.
- Delivery speed: separate teams can release a capability without rebuilding the whole application.
- Maintainability: service interfaces allow components to evolve, although distributed systems add monitoring and debugging complexity.

3. Hosting, Traffic Management, Caching and CDN

Hosting strategy

Amazon.com is operated by Amazon as a global service. AWS is Amazon's cloud-computing business, but it is important not to assume that every Amazon.com production component is simply hosted as a customer workload on public AWS. Amazon has operated large-scale infrastructure and data centres, and public sources do not provide a complete map of which retail components run on which infrastructure. A careful report should say that Amazon operates its own global infrastructure and uses a broad internal platform; exact placement of every Amazon.com component is not fully public.

How traffic and scaling are managed

1. Route traffic to healthy entry points and service instances using DNS, edge routing and load-balancing techniques.
2. Add or remove capacity as request volume changes; scale high-demand capabilities independently where the service design permits.
3. Isolate workloads and dependencies so a sudden spike or failure is less likely to spread across the whole system.
4. Monitor latency, errors, saturation and dependency health; use alarms and operational runbooks to respond to incidents.
5. Use asynchronous events for work that does not need to finish before the customer receives a response.

Caching and CDN

Caching stores frequently requested content or computed results closer to where they are needed. A cache can reduce response time, backend load and cost, but it introduces trade-offs: stale data, cache misses, invalidation and the possibility that a cold or unavailable cache sends too much traffic to the underlying database. Amazon's Builders' Library explicitly discusses these challenges and recommends measuring cache hit rate, selecting expiration policies deliberately, monitoring cache health and ensuring services remain resilient when a cache is unavailable.

A content delivery network (CDN) is especially useful for static assets such as images, stylesheets, scripts and other cacheable content. Edge delivery brings content closer to users and can reduce origin traffic. AWS documentation describes CloudFront as a CDN and shows how it can cache static content, while a web application firewall can help protect the public entry point. These are established AWS design patterns; they should not be presented as proof that every Amazon.com request follows the exact reference design.

Security and reliability

A retail platform must protect accounts, addresses and payment-related workflows. Common controls include encrypted transport (HTTPS/TLS), access control, monitoring, abuse protection, secure secrets handling, backups and recovery planning. Multi-zone deployment and redundant components can improve availability, while rate limits and workload isolation can help protect dependencies during traffic surges.

4. Unique Engineering Practices, Summary Table and Conclusion

Notable engineering ideas

• Service ownership and autonomy: Amazon's public eCommerce Foundation description highlights core systems and infrastructure that support teams building Amazon Stores experiences.
• Builders' Library: Amazon shares practical engineering guidance on distributed systems, caching, workload isolation, and operating reliable services.
• Workload isolation: techniques such as quotas, throttling and partitioning can reduce the chance that one high-volume workload harms others.
• Event-driven workflows: events allow downstream processes to run independently of the customer-facing request where appropriate.
• Observability and operational discipline: metrics, alarms, testing and incident response are essential when a user request crosses many services.

Summary table

Topic	Case-study summary
Application	Amazon.com — global e-commerce website
Architecture	Distributed, service-oriented / microservices-style; diagram is conceptual
Web server	Single product not publicly confirmed for the entire site; describe a distributed web/API entry layer
Backend stack	Many services and technologies; exact complete language/framework inventory is not public
Hosting	Amazon-operated global infrastructure; exact component-by-component placement is not fully public
Scaling	Load distribution, independent service capacity, workload isolation and operational monitoring
Caching / CDN	Caching reduces latency and backend load; CDN edge caching is a common delivery pattern
Reliability	Redundancy, dependency isolation, monitoring, alarms and recovery planning
Unique resources	Amazon Builders' Library; eCommerce Foundation engineering organisation

Conclusion

Amazon.com illustrates why large e-commerce platforms are better understood as distributed systems than as a single web server connected to one database. Separating business capabilities can support targeted scaling, independent releases and fault isolation. Caching, edge delivery, monitoring and workload isolation help improve performance and reliability, but each adds design and operational trade-offs. Because Amazon's complete internal implementation is proprietary, the strongest case study combines verified public statements with clearly labelled conceptual architecture rather than guessing exact servers or frameworks.

5. References

Thefollowing publicsources support the architecture concepts and the statements about Amazon's engineering environment. Accessed 10 October 2026.

1. Amazon Jobs — eCommerce Foundation. Describes core systems, services and infrastructure supporting Amazon Stores.
https://www.amazon.jobs/content/en/teams/e-commerce-foundation

2. Amazon Builders' Library — Caching Challenges and Strategies. Discusses cache benefits, consistency, expiration, monitoring and resilience when caches fail.
https://aws.amazon.com/builders-library/caching-challenges-and-strategies/

3. AWS — What are Microservices? Explains independent services, interfaces and the benefits and trade-offs of microservices architectures.
https://aws.amazon.com/microservices/

4. AWS Documentation — Guidance for Web Store on AWS. Reference design covering DNS, CloudFront, static content, web security and availability patterns for e-commerce applications.
https://docs.aws.amazon.com/solutions/web-store-on-aws/

5. AWS — Caching Overview. Explains how caching improves data retrieval performance and can reduce backend workload.
https://aws.amazon.com/caching/

6. AWS Architecture Blog — Data Caching Across Microservices in a Serverless Architecture. Describes cache-aside patterns, ElastiCache, DynamoDB Accelerator and event-driven integration examples.
https://aws.amazon.com/blogs/architecture/data-caching-across-microservices-in-a-serverless-architecture/

Academic note: The architecture diagram and explanatory examples are original educational illustrations based on the public sources above. They are not an official Amazon.com infrastructure diagram. The report avoids presenting undocumented server products or programming languages as confirmed facts.